

ReviewMind

Semantic AI Code Reviewer for GitHub

CSE 4011 – Intelligent Developer Tools and AI DevOps Workflows | Major Project Proposal

Problem Statement

Code review is essential for catching bugs, security issues, and missing error handling before merge, but it is slow, inconsistent, and dependent on reviewer availability. Small teams and student projects often skip thorough review entirely due to time constraints, letting preventable issues reach production. There is a need for an automated reviewer that understands code semantically — not just syntactically — and provides consistent, context-aware feedback the moment a change is proposed.

Objective

To build an AI-powered GitHub Action that automatically reviews every pull request using semantic code understanding and retrieval-augmented generation, flags bugs/security/error-handling issues, suggests relevant test cases, and posts the review as inline PR comments — with zero manual triggering, integrated natively into the developer's existing GitHub workflow.

How It Works

- 1. Trigger** – A GitHub Action fires automatically whenever a PR is opened or updated.
- 2. Fetch & Parse** – The PR diff is pulled via the GitHub API; changed files are parsed with tree-sitter into functions/classes.
- 3. Semantic Context** – Sourcegraph indexes the full repo so the bot can check how changed code connects elsewhere; LangChain/LlamaIndex retrieves relevant surrounding context (RAG).
- 4. Review Generation** – An LLM (via prompt-engineered reviewer persona) analyzes the diff + retrieved context, flags issues, and suggests missing test cases.
- 5. Post Feedback** – Review comments and test suggestions are posted back to the PR via the GitHub API as inline comments.

Tech Stack

Layer	Technology	Purpose
Trigger / CI	GitHub Actions	Auto-runs the bot on every PR event
Code Fetch	GitHub REST API / PyGithub	Pulls PR diff and changed files
Code Parsing	tree-sitter	Breaks changed files into functions/classes
Semantic Navigation	Sourcegraph (self-hosted, Docker)	Repo-wide semantic search for related code
Vector Store	ChromaDB / FAISS	Stores embeddings for retrieval
RAG Orchestration	LangChain / LlamaIndex	Retrieves relevant context before review generation
LLM	Groq API (Llama 3.3)	Generates review comments and test suggestions

Backend	FastAPI	Orchestrates fetch → retrieve → review → post pipeline
Posting	GitHub REST API	Writes inline PR comments

Alignment with Course Evaluation Phases

Phase	Weightage	Covered By
A1 – Code Understanding & Navigation	30%	Sourcegraph semantic navigation setup; prompt-engineered reviewer persona for code analysis
A2 – Tool Integration & Automation	30%	LangChain/LlamaIndex RAG for context retrieval; AI-generated test suggestions; GitHub Actions automation
A3 – End-Term Final Project	40%	Full pipeline demoed live on a real PR — trigger, review, and comments generated end to end

Why This Project

- Covers every rubric line item natively — no gaps, no bolt-on features needed.
- Read-only, comment-only design — no risky code execution or file editing, so the live demo cannot break the codebase.
- Fully demoable live: open a real PR in front of faculty and watch the bot review it in real time.
- Integrates directly into a real developer workflow (GitHub PRs) rather than a standalone tool most reviewers never open.